

```

# frontend/app.py
import streamlit as st
import requests
import datetime

st.set_page_config(page_title="SisCuratela Pro - Gestão e Compliance",
page_icon="🏛️", layout="wide")

# Inicialização de Sessão
if "logado" not in st.session_state:
    st.session_state.update({
        "logado": False,
        "token": None,
        "nome_usuario": None,
        "perfil_usuario": None
    })

# Função de Autenticação via API (FastAPI)
def login_backend(email, senha):
    url_api = "http://localhost:8000/auth/login"
    try:
        resposta = requests.post(url_api, json={"email": email, "senha": senha})
        if resposta.status_code == 200:
            dados = resposta.json()
            return True, dados["nome"], dados["perfil"], dados["access_token"]
        return False, None, None, None
    except requests.exceptions.ConnectionError:
        st.error("Erro Crítico: Falha de comunicação com o Servidor de Autenticação (Backend offline).")
        return False, None, None, None

# Função para Enviar Lançamento Financeiro para o Backend
def enviar_lancamento(dados_transacao, token):
    url_api = "http://localhost:8000/transacoes/novo"
    headers = {"Authorization": f"Bearer {token}"}
    try:
        resposta = requests.post(url_api, json=dados_transacao, headers=headers)
        if resposta.status_code == 200:
            return True, resposta.json().get("mensagem")
        else:
            erro_detalhe = resposta.json().get("detail", "Erro desconhecido no servidor.")
            return False, erro_detalhe
    except requests.exceptions.ConnectionError:
        return False, "Falha de conexão com o Backend."

# Tela de Login
if not st.session_state["logado"]:
    col1, col2, col3 = st.columns([1,2,1])
    with col2:
        st.markdown("<h1 style='text-align: center; color: #1f77b4;'>🏛️  
SisCuratela Pro</h1>", unsafe_allow_html=True)
        st.markdown("<p style='text-align: center; color: gray;'>Sistema de  
Gestão e Prestação de Contas (MPDFT)</p>", unsafe_allow_html=True)

```

```

st.write("----")

with st.form("form_login"):
    email = st.text_input("E-mail de Acesso (Allan, Itamar ou Dr.
Nelson)")
    senha = st.text_input("Senha de Acesso", type="password")
    submit = st.form_submit_button("Autenticar via Servidor Seguro")

    if submit:
        sucesso, nome, perfil, token = login_backend(email, senha)
        if sucesso:
            st.session_state.update({
                "logado": True,
                "nome_usuario": nome,
                "perfil_usuario": perfil,
                "token": token
            })
            st.rerun()
        else:
            st.error("Falha na autenticação. Verifique suas
credenciais.")

# Painel Logado (Enterprise-grade)
else:
    st.sidebar.title("Painel de Controle")
    st.sidebar.markdown(f"**Operador:** {st.session_state['nome_usuario']}")
    st.sidebar.markdown(f"**Perfil:** `{st.session_state['perfil_usuario']}`")
    st.sidebar.write("----")

    menu = st.sidebar.radio("Navegação", ["Painel Central", "Novo Lançamento
Financeiro"])

    if st.sidebar.button("Encerrar Sessão (Logout)"):
        st.session_state.clear()
        st.rerun()

    if menu == "Painel Central":
        st.title("🏢 Painel Executivo e Compliance")
        st.success("Conexão estabelecida com segurança entre Frontend, Backend e
PostgreSQL!")
        st.info("Utilize o menu lateral para registrar transações financeiras
com validação automática para a prestação de contas.")

    elif menu == "Novo Lançamento Financeiro":
        st.title("💰 Novo Lançamento Financeiro")
        st.markdown("Preencha os dados abaixo. O motor de compliance validará as
exigências do MPDFT antes de gravar no banco de dados.")

    # Inicializa o contador para limpar a memória do navegador após cada
salvamento
    if "form_counter" not in st.session_state:
        st.session_state["form_counter"] = 0

    with st.form("form_transacao"):

```

```

col_a, col_b = st.columns(2)

with col_a:
    id_conta = st.text_input("ID da Conta Bancária (UUID)",
value="conta-bb-principal")
    # Chave dinâmica que muda a cada envio, zerando o histórico do
navegador
    valor_digitado = st.text_input("Valor (R$)", placeholder="0,00",
key=f"valor_dinamico_{st.session_state['form_counter']}")

    with col_b:
        data_transacao = st.date_input("Data da Transação",
value=datetime.date.today())
        documento_ref = st.text_input("Documento de Referência (Nota
Fiscal / Recibo)")

        id_micro = st.text_input("ID da Categoria Micro (Ex: Farmácia /
Insumos)", value="micro-farmacia")
        descricao = st.text_area("Descrição Detalhada do Gasto (Proibido
termos genéricos como 'diversos')")

        submit_transacao = st.form_submit_button("Validar e Salvar")

if submit_transacao:
    import re

    # Trava de Compliance: Proíbe estritamente letras
    if re.search(r'[a-zA-Z]', valor_digitado):
        st.error("Rejeitado pelo Compliance: O campo de valor não pode
conter letras ou termos inválidos.")
        st.stop()

    # Trava de Compliance: Rigor estrito de centavos (máximo de 2 casas
decimais)
    if ',' in valor_digitado:
        partes = valor_digitado.split(',')
        if len(partes) > 2:
            st.error("Rejeitado pelo Compliance: Formato de valor
inválido (múltiplas vírgulas).")
            st.stop()
        if len(partes[1]) > 2:
            st.error("Rejeitado pelo Compliance: O valor não pode ter
mais de duas casas decimais (centavos).")
            st.stop()

    try:
        # Limpeza e conversão garantindo o padrão monetário
        valor_limpo = valor_digitado.replace(".", "").replace(",", ".")
        valor_float = float(valor_limpo)
        valor_float = round(valor_float, 2)

    except ValueError:
        st.error("Rejeitado pelo Compliance: Formato de valor inválido.
Utilize o padrão 150,00 ou 1.500,00")

```

```
st.stop()

payload = {
    "id_conta": id_conta,
    "id_usuario": "uuid-allan-001",
    "id_micro": id_micro,
    "data_transacao": str(data_transacao),
    "valor": valor_float,
    "descricao": descricao,
    "documento_referencia": documento_ref
}

sucesso_transacao, msg = enviar_lancamento(payload,
st.session_state["token"])
if sucesso_transacao:
    st.success(msg)
    st.session_state["form_counter"] += 1
    st.rerun()
else:
    st.error(f"Rejeitado pelo Compliance: {msg}")
```